

TP - Introduction to CUDA Programming

Oguz Kaya
oguz.kaya@universite-paris-saclay.fr

Part 1

Hello World in CUDA

Ex. 1

- a) Write a program `hello-cuda.cu` (skeleton code given) that launches the provided `cudaHello` CUDA kernel (or function).
 - In the CUDA kernel, print the block id and thread id of each thread, as well as the total number of blocks and threads/block (see the “Hello World, revisited” slide of the lecture for the four built-in variables).
 - Run this kernel using 64 threads. Try different number of blocks for all powers of 2 (1, 2, 4, 8, 16, 32, 64) and set the number of threads proportionally (64, 32, 16, 8, 4, 2, 1).

Part 2

CPU/GPU memory transfer using CUDA

In GPU programming model, the GPU device is used as an *accelerator* for the CPU that executes the main code and holds the data for the computation. Whenever a task for GPU is encountered, the CPU first transfers the data to the GPU, then executes the GPU kernel, finally transfers the results back to the CPU’s main memory. The goal of this exercise is to perform such transfers for statically (as `float A[1000]` for instance) and dynamically allocated arrays (through `malloc()/new()` or `std::vector` for instance).

In the given skeleton codes `cuda-copy-static.cu` and `cuda-copy-dynamic.cu`, the goal is to copy the CPU array `A[N]` into the GPU array `dA[N]` first, then copy the GPU array `dA[N]` into the CPU array `B[N]`. You will need to allocate the GPU array `dA[N]` statically or dynamically in these two codes, respectively.

Ex. 2

- a) Allocate a static/dynamic array `dA` of size `N` on the GPU. In the static case, you will do this by adding the keyword `__device__` in front of the array declaration, whereas in the dynamic case you will need to call `cudaMalloc()`.
- b) Copy the CPU array `A` into the GPU array `dA` using the corresponding CUDA `memcpy` function (remember, there are two `memcpy` functions, one for the static, other for the dynamic case).
- c) Copy the GPU array `dA` back to the CPU array `B` using the corresponding CUDA `memcpy` function.

Part 3

Writing a GPU-GPU `memcpy` kernel

In this exercise, you will write your own `memcpy` kernel to copy a GPU array into another GPU array. You will use this to perform the memory transfer `A[N] -> dA[N] -> dB[N] -> B[N]`, where `A[N]`, `B[N]` are CPU arrays and `dA[N]`, `dB[N]` are their GPU counterparts. You should use the given skeleton code `cuda-copy-kernel.cu` for this exercise.

Ex. 3

- a) First, complete the provided kernels `cudaCopyByBlocks` and `cudaCopyByBlocksThreads`.
- b) Then, perform the necessary memory allocations for `dA[N]` and `dB[N]`, then do the memory transfers using `cudaCopyByBlocks` kernel, as demanded in the skeleton code’s comments.
- c) Finally, do the same but using the `cudaCopyByBlocksThreads` kernel this time.

Part 4

CUDA saxpy kernel

In this exercise, you will write a saxpy BLAS kernel that performs the operation $y = ax + y$ for vectors x, y of size N and the scalar a . You will write multiple kernels that perform this operation:

Ex. 4

- a) First, a kernel that only launches blocks, and one thread per block, each block working on one vector element
- b) Another kernel that uses a certain number of threads (multiple of 32) per block, each thread working on one vector element
- c) Finally, a kernel that uses a certain number of threads per block, each thread working on K elements of the vector.

In doing these operations, you should also perform necessary memory copies at appropriate places as indicated in the provided skeleton code **saxpy.cu**.

Part 5

CUDA convolution kernel

In this exercise, you will write a simple 1D convolution kernel that performs the operation $y[i] = (x[i - 1] + x[i] + x[i + 1])/3.0$ for vectors x, y of size N and for all $1 \leq i < N$ (and set $y[0] = x[0], y[N - 1] = x[N - 1]$). You will write multiple kernels that perform this operation:

Ex. 5

- a) First, a kernel that only launches blocks, and one thread per block, each block working on one vector element
- b) Another kernel that uses a certain number of threads (multiple of 32) per block, each thread working on one vector element
- c) Finally, a kernel that uses a certain number of threads per block, each thread working on K elements of the vector, where K is a compile-time constant/macro (e.g., `#define K 8`)

In doing these operations, you should also perform necessary memory copies at appropriate places as indicated in the provided skeleton code **convolution.cu**.